

Assignment 2

[1] "FROM: Group 32 (Andrew Li, Kevin Du, Sebastian Hyland)"

Setup

First, let us read the dataset into a data frame `df` :

```
df <- read.delim("ovarian.data", sep = ",", header = FALSE)
features <- c(
  "perimeter", "area", "smoothness", "symmetry", "concavity", paste(
    "protein", seq(1, 25),
    sep = ""
  )
)

names(df) <- c("cell_id", "diagnosis", features)
df$diagnosis <- as.factor(df$diagnosis)

head(df, n = 2)
```

```
  cell_id diagnosis perimeter  area smoothness symmetry concavity protein1
1 85382601      M    17.02 23.98    112.80    899.3   0.11970   0.1496
2 88147102      B    15.00 15.51     97.45    684.5   0.08371   0.1096
  protein2 protein3 protein4 protein5 protein6 protein7 protein8 protein9
1  0.24170   0.1203   0.2248  0.06382   0.6009   1.3980   3.999   67.78
2  0.06505   0.0378   0.1881  0.05907   0.2318   0.4966   2.276   19.88
  protein10 protein11 protein12 protein13 protein14 protein15 protein16
1  0.008268  0.03082   0.05042   0.01112   0.02102  0.003854   20.88
2  0.004119  0.03207   0.03644   0.01155   0.01391  0.003204   16.41
  protein17 protein18 protein19 protein20 protein21 protein22 protein23
1    32.09    136.1    1344.0   0.1634   0.3559   0.5588   0.1847
2    19.31    114.2     808.2   0.1136   0.3627   0.3402   0.1379
  protein24 protein25
1    0.3530   0.08482
2    0.2954   0.08362
```

Now, we can load the required libraries:

```
library(ggplot2)
library(tidyverse)
library(plotROC)
library(RSNNS)

# For plot themeing
library(ggsci)
jco_palette <- rep(pal_jco()(10), length.out = 100)
jco_fill <- scale_fill_manual(values = jco_palette)
jco_colour <- scale_colour_manual(values = jco_palette)
```

Questions

Q1.1

To perform PCA on the dataset features, we use `prcomp` :

```
pca <- prcomp(
  df[features],
  scale. = TRUE, center = TRUE
)
pca_summary <- summary(pca)
str(pca_summary)
```

```
List of 6
 $ sdev      : num [1:30] 3.58 2.29 1.62 1.37 1.25 ...
 $ rotation  : num [1:30, 1:30] -0.22 -0.11 -0.229 -0.222 -0.137 ...
 .. attr(*, "dimnames")=List of 2
 .. ..$ : chr [1:30] "perimeter" "area" "smoothness" "symmetry" ...
 .. ..$ : chr [1:30] "PC1" "PC2" "PC3" "PC4" ...
 $ center    : Named num [1:30] 14.1809 19.3922 92.1982 663.7854 0.0965 ...
 .. attr(*, "names")= chr [1:30] "perimeter" "area" "smoothness" "symmetry" ...
 $ scale     : Named num [1:30] 3.5715 4.2746 24.1993 354.8356 0.0142 ...
 .. attr(*, "names")= chr [1:30] "perimeter" "area" "smoothness" "symmetry" ...
 $ x         : num [1:625, 1:30] -4.476 0.448 1.916 1.874 -2.802 ...
 .. attr(*, "dimnames")=List of 2
 .. ..$ : NULL
 .. ..$ : chr [1:30] "PC1" "PC2" "PC3" "PC4" ...
 $ importance: num [1:3, 1:30] 3.582 0.428 0.428 2.287 0.174 ...
 .. attr(*, "dimnames")=List of 2
 .. ..$ : chr [1:3] "Standard deviation" "Proportion of Variance" "Cumulative Proportion"
 .. ..$ : chr [1:30] "PC1" "PC2" "PC3" "PC4" ...
 - attr(*, "class")= chr "summary.prcomp"
```

By slicing the `importance` field of the `pca_summary` object, we can see statistics on each PC. The second row contains the variance associated, and the column is the PC of choice:

```
pca_summary_importance <- pca_summary$importance
pca_summary_importance[, 1:5]
```

	PC1	PC2	PC3	PC4	PC5
Standard deviation	3.581962	2.287258	1.623949	1.374095	1.249103
Proportion of Variance	0.427680	0.174380	0.087910	0.062940	0.052010
Cumulative Proportion	0.427680	0.602070	0.689970	0.752910	0.804920

As such:

```
cat(pca_summary_importance[2, 1] * 100, "%")
```

```
42.768 %
```

of variance is accounted for by PC1.

Q1.2

We can use the “Cumulative Proportion” row to find the first PC for which this value exceeds 90%:

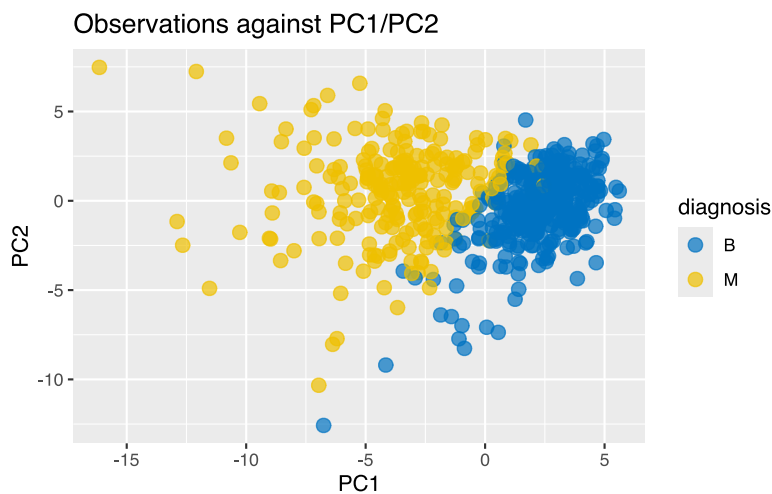
```
which(
  pca_summary$importance[3, ] ≥ 0.9
)[1] %>%
  cat(
    "First item with cumulative variance over threshold: PC",
    .,
    sep = ""
  )
```

First item with cumulative variance over threshold: PC9

As such, 9 dimensions are required to preserve 90% of the data’s variability.

Q1.3

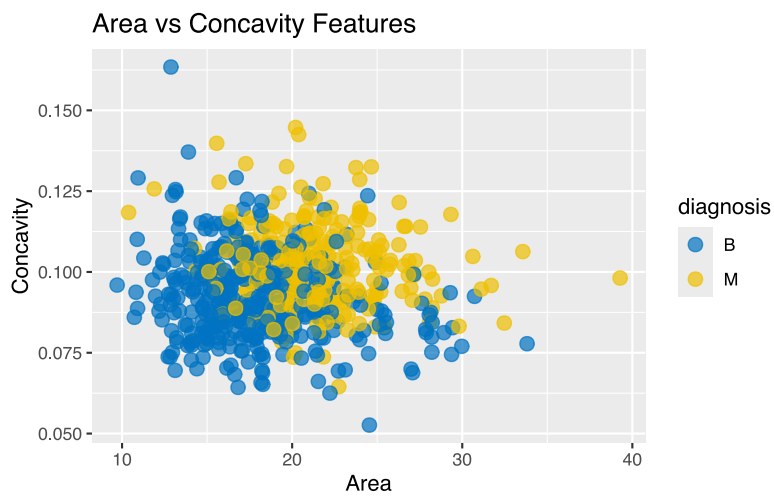
```
pca$x |>
  as.data.frame() |>
  select(PC1, PC2) |>
  mutate(diagnosis = df$diagnosis) |>
  ggplot(aes(x = PC1, y = PC2, color = diagnosis)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(
    title = "Observations against PC1/PC2",
    x = "PC1",
    y = "PC2"
  ) +
  jco_colour
```



Here, observations are plotted based on PC1 and PC2, coloured by diagnosis. Reasonably clear separation between the two clusters is visible.

Q1.4

```
df |>
  as.data.frame() |>
  ggplot(aes(x = area, y = concavity, color = diagnosis)) +
  geom_point(size = 3, alpha = 0.7) +
  labs(
    title = "Area vs Concavity Features",
    x = "Area",
    y = "Concavity"
  ) +
  jco_colour
```



Here, observations are plotted based on area and concavity. Although clustering is somewhat observed, it is not clear.

Q1.5

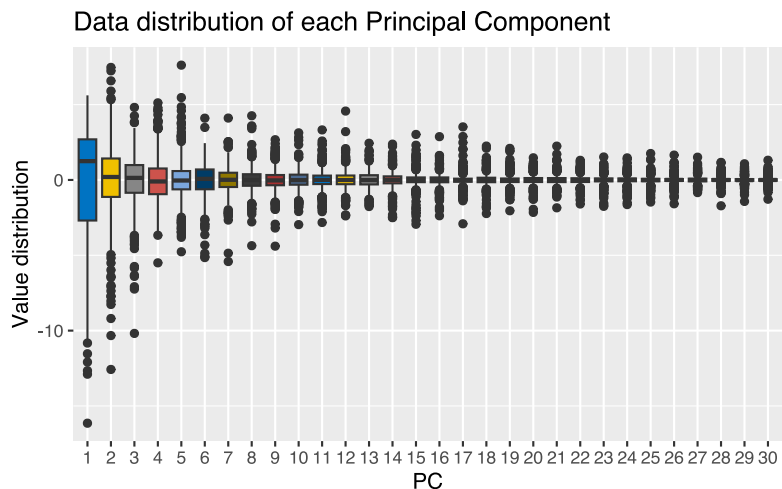
In [1.3](#), principal component analysis (PCA) transforms the original 30 features into a new set of dimensions (principal components), ordered by the quantity of variance that they capture (as such, the first and second PCs capture the most variance). On the other hand, concavity and area in [1.4](#) are just two of these raw features. Because PCA maximizes variance capture in each of its dimensions, response separation (in this case **diagnosis**) is typically greater than in individual raw features. This would be especially true if neither area nor concavity are dominant discriminatory features.

Following this theoretical basis, we can visually identify greater separation based on diagnosis in the [PCA chart](#) than the [concavity/area raw feature chart](#).

Q1.6

Let us first create boxplots which represent the data distribution of each PC:

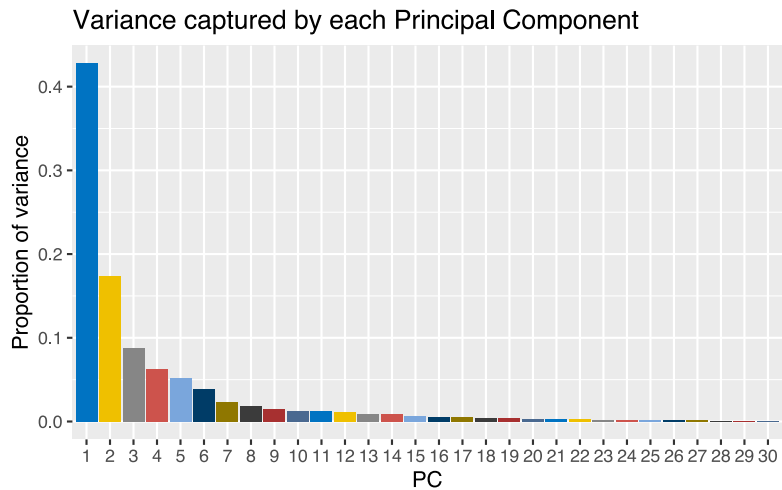
```
pca$x |>
  as.data.frame() |>
  pivot_longer(
    cols = everything(),
    names_to = "PC",
    values_to = "Value"
  ) |>
  mutate(PC = factor(parse_number(PC))) |>
  ggplot(aes(x = PC, y = Value, fill = PC)) +
  geom_boxplot(show.legend = FALSE) +
  labs(
    title = "Data distribution of each Principal Component",
    x = "PC",
    y = "Value distribution"
  ) +
  jco_fill
```



Now, let us visually represent the proportion of variance captured by each PC:

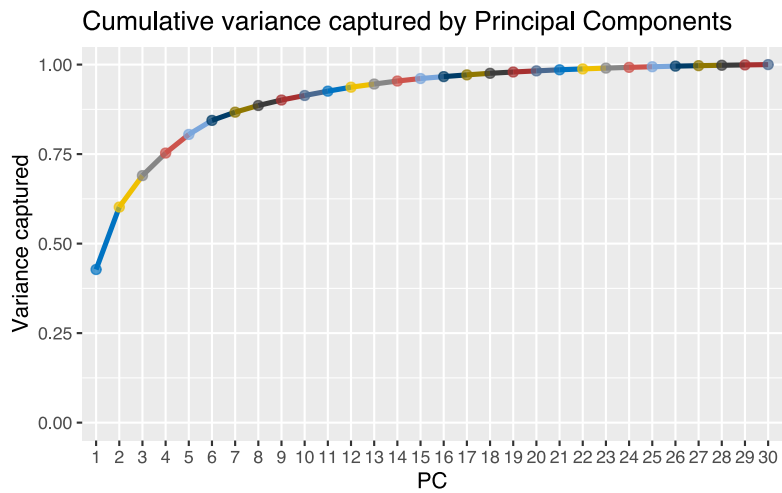
```
pca_plotting_summary <- pca_summary$importance |>
  t() |>
  as.data.frame() |>
  rownames_to_column("PC") |>
  mutate(PC = factor(parse_number(PC)))

ggplot(pca_plotting_summary,
  aes(x = PC, y = `Proportion of Variance`, fill = PC)
) +
  geom_col(show.legend = FALSE) +
  labs(
    title = "Variance captured by each Principal Component",
    x = "PC",
    y = "Proportion of variance"
  ) +
  jco_fill
```



Or, cumulatively:

```
ggplot(pca_plotting_summary,
  aes(x = PC, y = `Cumulative Proportion`, color = PC, group = 1)
) +
  geom_line(linewidth = 1.2, show.legend = FALSE) +
  geom_point(size = 2, alpha = 0.7, show.legend = FALSE) +
  labs(
    title = "Cumulative variance captured by Principal Components",
    x = "PC",
    y = "Variance captured"
  ) +
  coord_cartesian(ylim = c(0, 1)) +
  jco_colour
```



Q2

Let us first define the following helper functions:

```
metrics <- function(matrix) {
  true_pos <- matrix["M", "M"]
  true_neg <- matrix["B", "B"]
  false_pos <- matrix["M", "B"]
  false_neg <- matrix["B", "M"]

  accuracy <- (true_pos + true_neg) / sum(matrix)
  precision <- true_pos / (true_pos + false_pos)
  recall <- true_pos / (true_pos + false_neg)

  list(
    accuracy = accuracy,
    precision = precision,
    recall = recall
  )
}

print_metrics <- function(metrics) {
  cat("Metrics:\n")
  for (name in names(metrics)) {
    val <- metrics[[name]]
    cat(name, ":", val, "\n")
  }
}

print_confusion <- function(confusion_matrix) {
  cat("Confusion matrix:", "\n")
  print(confusion_matrix)
}
```

Q2.1

Let us define a function to execute k-means analysis, using the `kmeans` function:

```
kmeans_analysis <- function(features, scaled = TRUE, echo = TRUE) {  
  clusters <- (if (scaled) scale(features) else features) |>  
    kmeans(centers = 2) |>  
    pluck("cluster")  
  
  actual <- df$diagnosis  
  pred1 <- ifelse(clusters == 1, "M", "B")  
  pred2 <- ifelse(clusters == 1, "B", "M")  
  accuracy <- function(pred, actual) mean(pred == actual)  
  
  pred <- if (accuracy(pred1, actual) >= accuracy(pred2, actual)) {  
    pred1  
  } else {  
    pred2  
  }  
  
  table(pred, actual)  
}
```

Now, we can call this function to run k-means analysis on the dataset's feature columns:

```
analysis <- kmeans_analysis(df[features])  
print(analysis)
```

```
      actual  
pred  B   M  
B  371  35  
M   14 205
```

then compute and print metrics for this confusion table:

```
metrics(analysis) |>  
  print_metrics()
```

```
Metrics:  
accuracy : 0.9216  
precision : 0.9360731  
recall : 0.8541667
```

As such, this k-means model provides reasonably high accuracy when used to predict diagnoses from raw feature data, signifying reasonably high-quality data clustering.

Q2.2

We can repeat the above analysis 10 times using `replicate`:

```
replicate(
  10,
  { kmeans_analysis(df[features], echo = FALSE) |> metrics() },
  simplify = FALSE
) |>
  bind_rows() |>
  print()
```

```
# A tibble: 10 × 3
  accuracy precision recall
  <dbl>      <dbl>    <dbl>
1  0.922      0.936  0.854
2  0.922      0.936  0.854
3  0.922      0.936  0.854
4  0.922      0.936  0.854
5  0.922      0.936  0.854
6  0.922      0.936  0.854
7  0.922      0.936  0.854
8  0.922      0.936  0.854
9  0.922      0.936  0.854
10 0.922      0.936  0.854
```

K-means clustering is a stochastic algorithm, meaning its results can vary slightly between runs due to random initialization of cluster centroids. Different starting points can produce different local minima in the optimization process, leading to small variations in cluster membership and accuracy. Running the algorithm multiple times, as done here, helps mitigate randomness and ensures more stable performance estimates. By averaging results across multiple runs, we obtain a more reliable estimate of the model's true accuracy. Setting a random seed or using a large number of initializations (via the `nstart` parameter) can further reduce variability and improve reproducibility.

Q2.3

We can repeat k-means analysis but on principal components rather than features directly (the top 5 PCs are used):

```
analysis_pca <- pca$x[, 1:5] |>  
  kmeans_analysis(scaled = FALSE, echo = FALSE)  
print(analysis_pca)
```

```
      actual  
pred    B    M  
  B 371  35  
  M  14 205
```

```
analysis_pca |>  
  metrics() |>  
  print_metrics()
```

```
Metrics:  
accuracy : 0.9216  
precision : 0.9360731  
recall : 0.8541667
```

This is also reasonably accurate when used to predict diagnoses from principal components.

Q2.4

The results between Q2.2 and Q2.3 are very close: k-means clustering with all raw features achieved 92.16% accuracy, while clustering on the top 5 principal components (PCs) resulted in 91.84%. This suggests that just 5 PCs, capturing about 80% of the data's total variance, are sufficient to encode most of the class-separating information for successful clustering. However, the raw-feature model's slightly higher accuracy indicates that some subtle discriminative details may reside outside the top PCs and are not fully preserved by dimensionality reduction.

This trade-off highlights how PCA prioritizes directions of greatest variance rather than explicit class separation, sometimes omitting lower-variance features that contribute to clustering accuracy. Additionally, using more PCs — or carefully selecting which ones to include — could further improve performance and possibly match or exceed the raw-feature model. Practically, clustering on fewer PCs offers advantages in computational efficiency and easier model interpretation, especially in high-dimensional datasets. In cases where raw features contain noise or outliers, PCA can reduce their effect and potentially yield better or more stable clustering results.

Q3

Let us first extract training and testing datasets:

```
training_set <- df[
  sample(nrow(df))
  [1:(nrow(df) / 2)],
]
testing_set <- df[
  sample(nrow(df))
  [(nrow(df) / 2):(nrow(df))],
]
```

To keep factor ordering consistent:

```
diagnosis_order <- c("B", "M")
training_set$diagnosis <- factor(training_set$diagnosis, levels = diagnosis_order)
testing_set$diagnosis <- factor(testing_set$diagnosis, levels = diagnosis_order)
```

Q3.1

We will start by fitting a generalized linear model to the **training** set, then generate probabilities for the **testing** set:

```
set.seed(1)
glm_model <- training_set %>%
  # Train off training_set
  glm(
    data = .,
    formula = reformulate(
      names(.[features]),
      response = "diagnosis"
    ),
    family = binomial
  )
```

Now, we can convert these probabilities to diagnoses for each entry in the training and testing datasets, and compute metrics:

```
gen_preds <- function(x) ifelse(x > 0.5, "M", "B")

run_model_analysis <- function(model, slice_op, train, test, type = "response") {
  cat("----- Training analysis -----", "\n")

  if (is.null(type) || type == "") {
    training_preds <- predict(model, train[slice_op])
    testing_preds <- predict(model, test[slice_op])
  } else {
    training_preds <- predict(model, train[slice_op], type = type)
    testing_preds <- predict(model, test[slice_op], type = type)
  }

  training_confusion <- training_preds |>
    gen_preds() |>
    table(train$diagnosis) |>
    print_confusion()

  cat("\n")
  metrics(training_confusion) |>
    print_metrics()

  cat("\n", "----- Testing analysis -----", "\n", sep = "")
  testing_confusion <- testing_preds |>
    gen_preds() |>
    table(test$diagnosis) |>
    print_confusion()

  cat("\n")
  metrics(testing_confusion) |>
    print_metrics()
}

run_model_analysis(glm_model, features, training_set, testing_set)
```

```
----- Training analysis -----  
Confusion matrix:
```

	B	M
B	189	2
M	3	118

```
Metrics:  
accuracy : 0.9839744  
precision : 0.9752066  
recall : 0.9833333
```

```
----- Testing analysis -----  
Confusion matrix:
```

	B	M
B	192	9
M	5	107

```
Metrics:  
accuracy : 0.9552716  
precision : 0.9553571  
recall : 0.9224138
```

This model had perfect accuracy when predicting against the training dataset, and quite high (95.2%) accuracy when predicting against the testing dataset.

Q3.2

First, let us run PCA on the training and testing datasets:

```
pca_model <- prcomp(
  training_set[features],
  scale. = TRUE,
  center = TRUE
)

into_pca_df <- function(dataset) {
  pca_model |>
  predict(dataset[features]) |>
  as.data.frame() |>
  select(1:5) |>
  mutate(diagnosis = dataset$diagnosis) |>
  # Set diagnosis to first col, then 5 PC cols
  select(diagnosis = diagnosis, everything())
}

training_pca_df <- into_pca_df(training_set)
testing_pca_df <- into_pca_df(testing_set)
```

Which provides neat data frames:

```
head(training_pca_df)
```

	diagnosis	PC1	PC2	PC3	PC4	PC5
351	B	-3.817034	-0.5888836	2.0816962	-0.5862178	1.6872805
501	B	-1.265399	-0.8972561	-0.3303776	1.2104538	1.3803463
396	M	-2.138296	1.7543456	-1.6260727	-1.0468187	-0.2475568
21	M	3.787371	0.3042466	1.4549317	-0.8717029	0.2500570
93	B	-1.366248	-2.3015311	0.3199696	0.8255133	-0.6728590
352	B	-2.809597	1.5275293	0.2606637	-3.0070479	0.5128554

```
head(testing_pca_df)
```

	diagnosis	PC1	PC2	PC3	PC4	PC5
305	B	-1.0167984	-0.3148746	2.2477444	-0.1664782	-1.7399005
319	B	-2.4976800	-0.7582382	-1.7832700	2.2577921	1.2975429
149	M	0.5069569	-0.6834290	-2.5552464	-0.2856766	0.3847512
343	B	2.1455436	-3.9848548	2.2191192	0.7931248	-3.1817176
549	M	4.6042632	2.7678473	0.3959951	0.1662045	0.2330397
559	M	4.5075384	0.5586266	3.4409245	-1.3828719	-0.5651268

Now, we can repeat the analysis from [3.1](#) with the top 5 PCs:

```
glm_pca_model <- training_pca_df %>%
  glm(
    data = .,
    formula = reformulate(
      colnames(training_pca_df[2:ncol(training_pca_df)]),
      response = "diagnosis"
    ),
  ),
```

```
family = binomial
)
```

Finally, associate probabilities with diagnoses and generate metrics:

```
run_model_analysis(
  glm_pca_model,
  2:ncol(testing_pca_df),
  training_pca_df,
  testing_pca_df
)
```

```
----- Training analysis -----
Confusion matrix:
```

	B	M
B	187	3
M	5	117

```
Metrics:
accuracy : 0.974359
precision : 0.9590164
recall : 0.975
```

```
----- Testing analysis -----
Confusion matrix:
```

	B	M
B	194	7
M	3	109

```
Metrics:
accuracy : 0.9680511
precision : 0.9732143
recall : 0.9396552
```

This model has high but not perfect (98.0%) accuracy against its training dataset, and very high (96.8%) accuracy when predicting against the testing data.

Q3.4

The results in both [3.1](#) and [3.2](#) were both excellent, but using the top 5 PCs yielded marginally better accuracy when predicting diagnoses on testing data (96.8% vs 95.2%). PCA can help by reducing low variance noise in the data, which tends to help stabilize it. The model built off raw features has a perfect fit when run on its training data, which suggests that the model is overfit or has complete separation; it then performs worse on the test dataset. On the other hand, the model built off the top five PCs has slightly worse accuracy when rerun on its training data (98.0%) but then performs better on the test dataset.

Q3.5

Since the PC model outperformed the raw features model, let us use it on the dataset at large and generate a ROC curve. First, convert the original `df` into a suitable data frame, and obtain metrics:

```
df_pca <- into_pca_df(df)
model_preds <- predict(glm_pca_model, df_pca, type = "response")

model_confusion <- model_preds |>
  gen_preds() |>
  table(df_pca$diagnosis) |>
  print_confusion()
```

Confusion matrix:

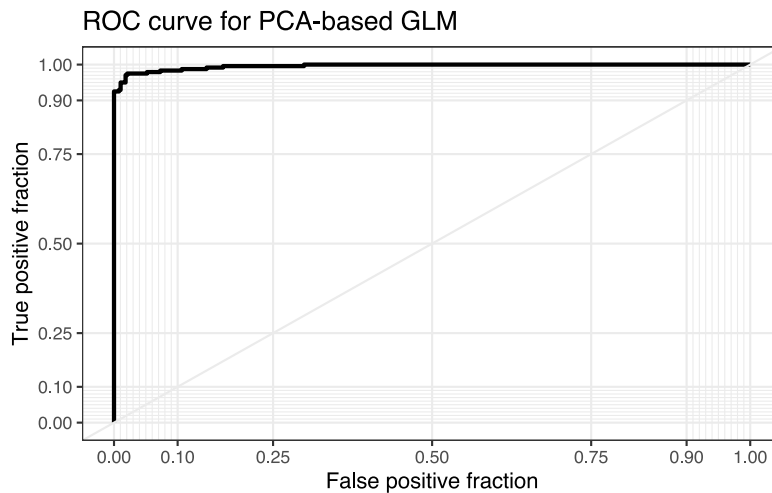
	B	M
B	378	10
M	7	230

```
metrics(model_confusion) |>
  print_metrics()
```

Metrics:
accuracy : 0.9728
precision : 0.9704641
recall : 0.9583333

Now, we can generate a ROC curve:

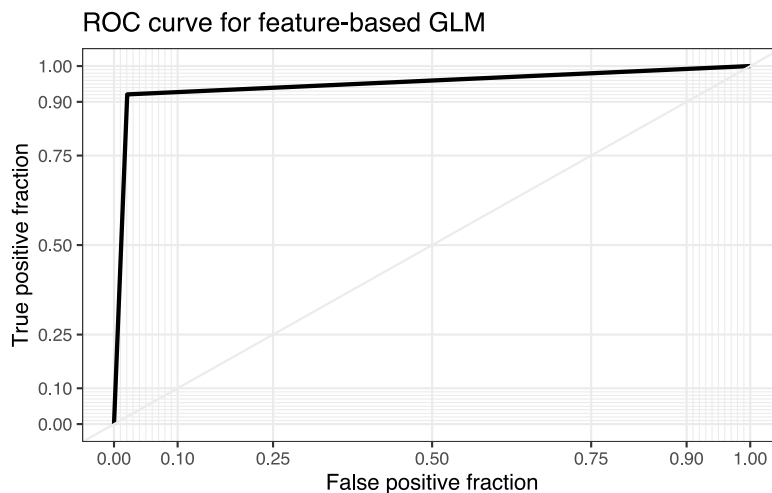
```
data.frame(
  D = as.numeric(df$diagnosis == "M"),
  M = model_preds
) |>
  ggplot(aes(d = D, m = M)) + geom_roc(n.cuts = 0) + style_roc() +
  labs(title = "ROC curve for PCA-based GLM")
```

The ROC curve bows sharply toward the top-left corner, demonstrating strong predictive power. This indicates high sensitivity (true positive rate) with minimal false positives across most threshold values, suggesting the model has excellent discrimination between benign and malignant cases. Unlike single-threshold metrics, the ROC curve provides a comprehensive view of the model's performance across all possible classification thresholds, illustrating the trade-off between sensitivity and specificity.

We can generate a similar curve for the model trained off raw features:

```
model_preds_raw <- predict(glm_model, df[features], type = "response")
data.frame(
  D = as.numeric(df$diagnosis == "M"),
  M = model_preds_raw
) |>
ggplot(aes(d = D, m = M)) + geom_roc(n.cuts = 0) + style_roc() +
  labs(title = "ROC curve for feature-based GLM")
```



Q3.6

Raw features

```
scaled_training_matrix ← scale(training_set[features])
ctr ← attr(scaled_training_matrix, "scaled:center")
sdr ← attr(scaled_training_matrix, "scaled:scale")

scaled_training_df ← training_set
scaled_training_df[features] ← scaled_training_matrix

scaled_testing_df ← testing_set
scaled_testing_matrix ← scale(testing_set[features], center = ctr, scale = sdr)
scaled_testing_df[features] ← scaled_testing_matrix

y_train_num ← as.numeric(training_set$diagnosis == "M")
mlp_all ← mlp(
  as.matrix(scaled_training_df[features]),
  y_train_num,
  size = 5,
  maxit = 200,
  learnFuncParams = c(0.1),
  linOut = FALSE
)

run_model_analysis(mlp_all, features, scaled_training_df, scaled_testing_df)
```

```
----- Training analysis -----
Confusion matrix:
```

	B	M
B	192	2
M	0	118

```
Metrics:
accuracy : 0.9935897
precision : 1
recall : 0.9833333
```

```
----- Testing analysis -----
Confusion matrix:
```

	B	M
B	197	7
M	0	109

```
Metrics:
accuracy : 0.9776358
precision : 1
recall : 0.9396552
```

PCA

```
y_train_num <- as.numeric(training_pca_df$diagnosis == "M")
mlp_pca <- mlp(
  as.matrix(training_pca_df[,2:6]),
  y_train_num,
  size = 5,
  maxit = 200,
  learnFuncParams = c(0.1),
  linOut = FALSE
)
run_model_analysis(mlp_pca, 2:6, training_pca_df, testing_pca_df)
```

----- Training analysis -----
Confusion matrix:

	B	M
B	192	4
M	0	116

Metrics:
accuracy : 0.9871795
precision : 1
recall : 0.9666667

----- Testing analysis -----
Confusion matrix:

	B	M
B	195	7
M	2	109

Metrics:
accuracy : 0.971246
precision : 0.981982
recall : 0.9396552